

Where the Time Goes: Profiling, Segmenting and Verifying CPU and GPU Rendering in a Narrated-Video Pipeline

Rajesh Kumar Kona
Independent researcher
Edinburgh, United Kingdom
konarajeshkumar011@gmail.com

Abstract

Automatic explainer-video systems draw every frame themselves, and rendering is the one cost that grows with the length of the video. I study where that time goes in Voice-to-Video, my narration-to-video pipeline, whose reference renderer composes frames with Pillow and encodes them with x264, and I ask how the work should be split between CPU and GPU. Measured per content kind at 1080p, composition takes 54% of CPU time for typography, 74% for mixed content and 85–90% for photographs and transitions; a profile of a real project put it at 77%. By Amdahl’s law that caps a hardware encoder at 1.30×, while removing redundant buffer copies alone took composition from 12 to 26 frames/s. Three designs follow: frame-exact segmented rendering, which makes renders resumable and parallel; a narrow OpenGL 3.3 compositor that offloads only photograph resampling; and an equivalence contract that admits a GPU only if no channel of any pixel differs from the CPU reference by more than 2 levels. The two-pass Lanczos-3 shader meets the contract on an NVIDIA GeForce GTX 1650 (23/23 scenes, worst difference 2) and on Mesa llvmpipe (worst 1). Ablations show the clamp between passes, dropped edge taps and the Lanczos kernel are each necessary (without them the worst differences are 23, 57 and 84). On the GTX 1650 at 1080p the GPU composes photographs 11.7× faster than the CPU (3.7 to 43.3 frames/s), transitions 7.7× and mixed content 3.0×; typography is 0.94× and stays on the CPU. Once photographs compose on the GPU, x264 becomes the larger share of the work (64%), so the case for hardware encoding has to be reopened. Two- and four-hour renders finished with no memory growth and zero duration error.

Keywords

video rendering, GPU compositing, image resampling, Lanczos, Amdahl’s law, checkpointing, differential testing, x264

1 Introduction

Systems that turn narration into video, such as lecture tools, explainer generators and automated editors, do most of their visual work themselves. They move photographs under slow camera motion, draw charts and typography, burn in captions, and blend shots with dissolves, wipes and pushes, all timed to one audio track. Upstream stages such as transcription, planning and asset retrieval cost roughly the same whatever the video’s length. Rendering does not: a forty-minute video at 30 frames/s is 72,000 frames, each composed and encoded.

Voice-to-Video (VTV) [10] is the pipeline I built. Its reference renderer is deliberately simple. A pure function describes each frame, Pillow [4] paints it, and x264 [21] (via FFmpeg [8]) encodes it. When renders became the bottleneck users felt (a forty-minute

video took about four times its own length), the obvious fix was a “fast” mode using a hardware encoder. This paper explains why I did not do that, what I did instead, how I checked that the GPU draws the same picture as the CPU, and how much faster it turned out to be. It makes five contributions:

- (1) **A per-content profile** of composition against encoding in a Pillow/x264 pipeline, measured on a project timeline and on four synthetic content kinds at 720p and 1080p, with the resulting Amdahl bounds (§4).
- (2) **Frame-exact segmented rendering**, in which checkpointing, resumption, parallelism and the unit of fallback come from one decomposition (§5).
- (3) **A reference-matching GPU resampler** that reproduces Pillow’s two-pass Lanczos-3 within two levels per channel, with ablations on the design choices that make it match (§6, §8.3).
- (4) **A zero-outlier equivalence contract** used to admit a GPU at run time and to diagnose failures. It found four defects on first contact with hardware, and pixel counts and bounding boxes explained each of them (§7, §8.1).
- (5) **Measured GPU throughput** per content kind on a consumer GPU, and what it does to the profile: after offloading, encoding becomes the larger cost for photographs (§8.5).

2 Background and Related Work

Parallel video processing. ExCamera [9] and Sprocket [2] split video work into many small independent tasks on cloud functions. ExCamera also shows how to keep compression efficiency when encoding in fine-grained chunks. My segmentation uses the same idea of independent units of work, as in MapReduce [5], on a single machine, where the units also serve as durable checkpoints in the sense of rollback recovery [7]. The difference is that each segment is *synthesised* from a scene description rather than decoded from an input video. This lets segment boundaries be chosen where the picture changes anyway.

Image pipelines on CPUs and GPUs. Halide [16] separates what an image pipeline computes from how it is scheduled on CPUs and GPUs. I make the same separation at a coarser grain: a display list is the *what*, and interchangeable painters are the *how*. Compositing semantics follow Porter and Duff [15]. Resampling quality depends on the reconstruction filter [12, 20]. Lanczos windows [6] are a common high-quality choice, and VTV’s reference resizes photographs with Pillow’s Lanczos filter (support 3).

Testing without an exact oracle. A GPU cannot promise bit-identical output to a CPU. Texture addressing is implementation-defined in OpenGL [17], and floating-point behaviour differs

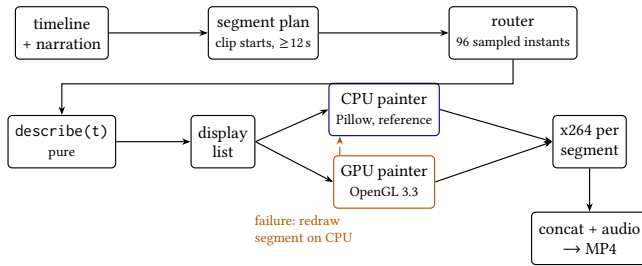


Figure 1: Render architecture. Both painters consume the same display list. Segments go to the CPU unless a sampled instant needs photograph resampling, and a failed GPU segment is redrawn by the reference painter.

between devices [23]. Comparing two implementations of the same specification is differential testing [11]. Choosing an acceptance rule when no exact oracle exists is the oracle problem [3]. Perceptual metrics such as SSIM [22] are designed to tolerate small, spread-out differences. I need the opposite: to reject small, *localised* ones, such as a caption one pixel too high.

3 The Pipeline

VTV is a Python 3.11 system of 173 source files (63,528 lines) with 1,708 test functions. Rendering consumes a resolved timeline of clips, captions and one narration file. Two early decisions constrain everything here. First, *the narration is the only clock*. Second, the renderer *draws every frame itself* rather than stitching encoded clips, so that transitions are a two-frame mix and there is no second timeline for audio to drift against.

Describe, then paint. The composer is split into a pure function and a painter (Figure 1). `describe(t)` returns a *display list*: ordered layers (background, picture, drawn primitive, plate, label, vignette, message), a separate list of overlays (captions, credits, badges) that transitions never mix, the outgoing side of any transition, and the eased mix progress. It touches no pixels, fonts or files. A *painter* turns a display list into an RGB frame. The invariant is that every painter consumes the same display list; if a backend needed its own description, the composer would have to know which backend it serves.

4 Where the Time Goes

Project profile. A profile of a real 1080p project timeline attributed 77% of render time to composition and 23% to x264. Glyph rasterisation was only 0.7% of composition. The cost lay in moving full-frame buffers: a paste, a blend, a channel split and a colour conversion per frame. Most of that work was redundant. Removing it raised composition from 12 to 26 frames/s on unchanged hardware, a 2.2× gain before any accelerator. Two further fast paths came from the same method. Single-layer frames skip an RGBA round trip worth about 10 ms of a 42 ms 1080p frame. A cached vignette mask removes 19 ms per frame, which amounts to 43 s over a 76-second video.

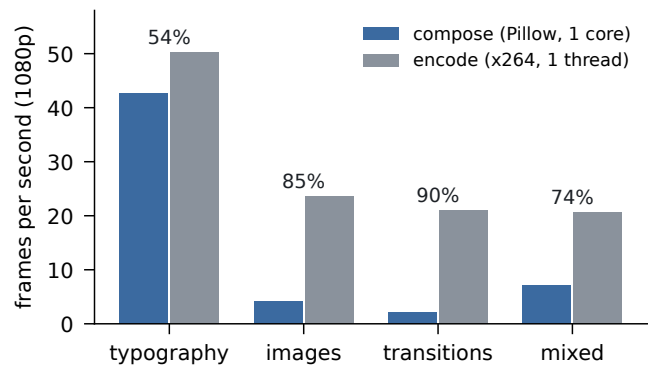


Figure 2: E1: frames/s for composition (one core) and x264 encoding (one thread) at 1080p. Labels give composition’s share of CPU time.

Per-content profile (E1). Composition cost depends on content, so I re-measured it on four synthetic workloads from the repository’s benchmark: animated typography; a photograph under a slow zoom; a dissolve between two photographs; and alternating typography and photograph-with-caption frames. The photograph is twice the frame size, so every picture frame is a reduction. For each workload I composed 24 frames with the reference painter on one core, then encoded the same frames with x264 (veryfast, CRF 23, one thread; the 24 frames are fed twice and the time, including encoder start-up, is halved). Each measurement is the median of three repeats. Figure 2 shows 1080p; 720p shares agree within one percentage point (54, 85, 90 and 73%). This run used a 2-vCPU cloud container (Intel Xeon at 2.80 GHz, FFmpeg 6.1.1), a different machine from the project profile. The *mixed* workload lands at 74%, close to the project’s 77%.

Consequences. By Amdahl’s law [1], with composition at 77%, an encoder of infinite speed yields at most $1/0.77 \approx 1.30\times$, while accelerating composition alone has a ceiling of $1/0.23 \approx 4.35\times$ (Figure 3). For photograph-heavy content the gap widens: at 85–90% composition, the encoder ceiling falls to 1.11–1.18×. A second argument was concurrency. Segments run one encoder per worker process, and GeForce cards historically capped concurrent hardware encode sessions at three, raised to five in 2023 [19]. NVIDIA’s current support matrix lists twelve [14]. That cap now exceeds the renderer’s eight workers, so the concurrency argument has weakened; the Amdahl bound has not. So VTV reports which hardware encoders the local FFmpeg can open, but keeps encoding on the CPU and spends the acceleration effort on composition. Section 8.5 shows that this decision has to be revisited once composition is fast.

5 Frame-Exact Segmented Rendering

The first renderer piped every frame into one FFmpeg process. It was simple and could not drift from its audio, but it was all-or-nothing and used one core. Threads could not help, because the global interpreter lock serialises the Python composition that dominates the profile.

Rule. A frame range that has been encoded is a file whose name says so. A segment is finished if and only if `seg-NNNNN.mp4` exists.

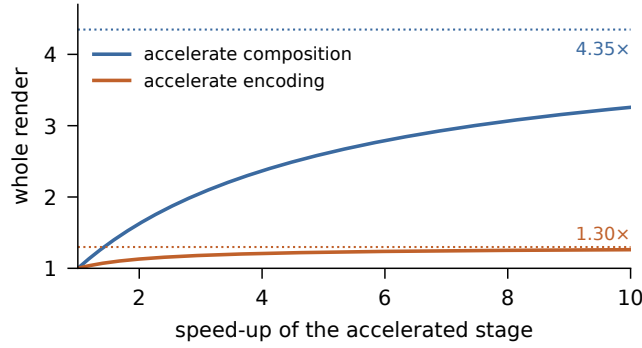


Figure 3: Whole-render speed-up predicted from the measured 77/23 profile when only one stage is accelerated (analytical).

That name is created only by an atomic rename from `.part.mp4` after the encoder exits with status zero. Resumption is not a separate recovery routine. Every render asks of every segment whether it is already done, and a first attempt is simply the case where the answer is always no.

Boundaries. Segments are cut only at clip starts, accumulating until a segment holds at least $\text{round}(12 \cdot \text{fps})$ frames. Twelve seconds bounds lost work, yields about two hundred segments for a forty-minute video, and keeps FFmpeg’s roughly 30 ms start-up under half a percent of each segment. Boundaries are computed in *frame indices*, $\text{round}(\text{start} \cdot \text{fps})$, and each segment runs to the next boundary. The segments therefore partition $\text{range}(\text{total_frames})$ exactly. Rounding each segment’s own duration instead would gain or lose a frame per segment, and a two-hundred-segment video would drift seconds from its narration.

Invalidation. The scratch directory is named by a SHA-256 fingerprint of the timeline and settings with only timestamp fields removed. A retry finds its own segments, and any edit lands in a fresh directory. The fingerprint deliberately over-invalidates: a needless re-render costs time, whereas reusing a stale segment ships a wrong picture.

Parallelism and assembly. Because `describe` is pure, frame n does not depend on frame $n - 1$, so segments are also the unit of parallel work. They are encoded independently (`-c:v libx264 -preset veryfast -threads 1`) in a process pool of $\min(8, \text{cores}, \text{outstanding})$ workers. The final file is a concat-demuxer stream copy, with the narration muxed in at that step. Tests compare decoded frames (`ffmpeg -f framemd5`) to show that a resumed render equals an uninterrupted one and a pooled render equals an inline one. A harness kills the renderer mid-job and fails if any finished segment is touched during the resume.

Scaling (E2). I rendered a 120-second typography video at 1080p (ten 12-second segments) on my GTX 1650 machine with 1, 2, 4 and 8 workers. It took 96, 62, 54 and 52 s: 1.55×, 1.78× and 1.85× faster than one worker. Ten equal segments cap those speed-ups at 2, 3.3 and 5 (with 8 workers the last two segments run alone), and each worker is really two busy processes, Python composition and

an x264 encoder. With 8 logical processors, four workers already ask for all eight, which probably explains why the gains flatten. I did not record per-core utilisation. Even one worker rendered faster than real time (0.80× the video’s length). On a 2-vCPU cloud container a 48-second render took 28.6 and 29.1 s with one worker and 27.5 s with two, as expected when one worker already fills both cores.

6 A Reference-Matching GPU Compositor

Division of work. The GPU painter uses OpenGL 3.3 through ModernGL [13] in a windowless context. OpenGL 3.3 is available across NVIDIA, AMD and Intel drivers, and the backend is a single module. Only one operation moves to the card: *resampling and placing photographs*. Text, plates, the animation engine, overlays and the transition mix stay with the reference painter, for the following reasons. Text is 0.7% of composition (measured), and rasterising glyphs with the reference on both paths makes captions bit-identical. Pillow’s RGBA drawing *replaces* pixels and then flattens onto the theme background, whereas a GPU alpha-composites; reproducing that quirk in a shader would reproduce a behaviour nobody chose. The reference wipe has a hard edge with a blurred band, and moving it would have cost three readbacks and two uploads to replace one blend. What remains is expensive and parallel: resampling a photograph into a moving frame, every frame of every picture shot.

Matching Pillow’s resampler. The shader evaluates

$$L_3(x) = \frac{3 \sin(\pi x) \sin(\pi x/3)}{\pi^2 x^2} \quad (0 < |x| < 3), \quad (1)$$

with $L_3(0) = 1$ and $L_3(x) = 0$ otherwise. It follows Pillow’s horizontal and vertical passes in three details that change pixels. First, output pixel i samples source coordinate $(i + 0.5) s$. Second, when reducing, the kernel is widened by $\max(1, s)$. Third, taps that fall outside the image are *dropped* and the surviving weights renormalised, rather than clamped to the edge. Pillow resizes in two one-dimensional passes and stores an 8-bit image between them. Lanczos overshoots at high-contrast edges, and that intermediate clamps the overshoot before the second pass. The GPU painter therefore renders pass one into an RGBA8 framebuffer with blending disabled, clamps its output, and reads it back in pass two. The loop is capped at 64 taps; reductions beyond 10× send the whole frame to the CPU. The remaining difference is arithmetic: Pillow uses 22-bit fixed-point weights and integer accumulation, while the shader uses float32.

Routing, capability and fallback. A display list *needs resampling* only if a picture is present, including on the outgoing side of a transition. The painter and the segment router share that one predicate. The router samples 96 instants plus the last frame of each segment and sends typography-only segments to the CPU. At start-up a probe initialises a context, compiles the shaders and *draws a calibration frame that must pass the contract of §7*. Only then is the device registered. The result is cached against a fingerprint of vendor, renderer and driver, so a driver update re-runs the probe. A GPU failure (lost context, out of memory, shader error) marks only that segment for redrawing on the next backend, and the job records how many segments each backend drew.

Table 1: Defects found on first contact with hardware.

Defect	Evidence
1. Framebuffer read bottom-up	7 of 8 failures; box = plate $\cup$ mirror
2. Cleared to black, not the theme background	18,432 px in picture-contained
3. Picture placed at its box, not the reference’s clamped cover box	remaining 2,304 px (a 9-row band)
4. Overlays alpha-composited, not replaced	overlay scenes off by 193

7 Defining “the Same Picture”

The acceptance rule has two parts. (1) No channel of any pixel may differ from the reference by more than 2 (on 0–255). (2) *No pixel may exceed that*: the number of out-of-tolerance pixels must be zero. Part 2 is what catches subtle errors. A mean-difference or perceptual test would accept a caption sitting one pixel high on every frame of a four-hour video. Gross errors, such as a missing overlay or a reversed wipe, differ by hundreds and fail either part. The calibration set has 23 scenes: background; message; four picture placements (fitted, zoomed, panned, contained); two plates; two text anchors; two vignette strengths; overlays over a picture and during a blend; and blend, wipe and push at 0, 50 and 100% progress. For each scene the harness reports the worst channel difference, the out-of-tolerance count, the mean absolute difference (MAD) and a bounding box.

A classifier explains failures by transforming the GPU frame until the difference collapses. It tries vertical and horizontal flips, ± 1 px shifts, a uniform offset (wrong clear colour), edge-only and high-frequency-only comparisons. Both the harness and the classifier were tested before they were trusted. Deliberately broken painters (a 2 px plate shift, dropped overlays, an exception) must fail, and two reference painters must pass.

8 Results

8.1 First Contact with Hardware

The first run on a GeForce GTX 1650 (OpenGL 3.3, driver 610.88, Windows build 26200) passed 15 of 23 scenes. The *pattern* of failures pointed to the cause. Every failing scene contained something vertically asymmetric (a plate near the top, a line of text); every passing scene was symmetric. A plate at rows 11–50 of a 180-row frame differed across rows 11–169, which is the plate plus its mirror. Table 1 lists the four defects. In one scene, defects 2 and 3 account for $18,432 + 2,304 = 20,736$ pixels, exactly the reported outlier count.

8.2 Equivalence on Two OpenGL Implementations (E3)

After the fixes, I re-ran the calibration set with a detailed test still containing thin high-contrast lines, a worst case for resampling. On the GTX 1650 all 23 scenes passed: worst difference 2, zero out-of-tolerance pixels, MAD at most 0.07. I then ran the unmodified painter and harness on Mesa’s llvmpipe software rasteriser (OpenGL 4.5, LLVM 20) [18], an independent implementation of the same API. All 23 scenes passed there too, with worst difference 1 and MAD below 0.001 (Figure 4). On both implementations, the

Table 2: E4 ablations on Mesa llvmpipe (23 scenes, detailed still).

Variant	Pass	Worst	Outlier px	Max MAD
Shipped (RGBA8 mid)	23	1	0	<0.001
Float intermediate, clamped	23	1	0	0.087
Float intermediate, no clamp	8	23	3,753	0.113
Edge-clamped taps	13	57	15,211	0.497
Bilinear texture filter	8	84	677,297	6.883

scenes without a photograph (background, message, plates, text, vignettes) were bit-identical. Every non-zero difference occurred in a resampled picture, consistent with the float32-versus-fixed-point residual. The benchmark in §8.5 repeated the check on the GTX 1650 at 640×360 with its own photograph: all 23 scenes passed again, with a worst difference of 1.

8.3 Ablations (E4)

To test which matching details are necessary, I built variants that each differ from the shipped painter, or from the preceding variant, in one choice, and re-ran the calibration set on llvmpipe (Figure 5, Table 2). Keeping a floating-point intermediate but still clamping between passes *also passes*. The clamp is therefore what matters for the tolerance, and 8-bit quantisation only lowers the mean difference (MAD 0.087 versus below 0.001). Removing the clamp as well, so that Lanczos overshoot reaches the second pass as in a single floating-point pass, fails 15 scenes with a worst difference of 23. Clamping out-of-range taps to the edge texel fails 10 scenes with a worst difference of 57 along the borders. The texture unit’s bilinear filter fails every picture scene, with a worst difference of 84 and a MAD of 6.9. The eight scenes that pass under every variant are exactly those without a photograph. An earlier, failing diagnosis run on the GTX 1650, whose images are in the repository, shows worst differences of 11–23 across its 15 picture-bearing scenes on the same still. That is consistent with the no-clamp variant and with the code’s own account of its first, single-pass floating-point shader.

8.4 Long Renders

Before this work, the longest end-to-end render was 150 s. A long-render harness renders on the desktop agent with the GPU painter available. It samples resident memory (process and children), scratch disk and VRAM every two seconds, and ends each length by measuring the output with ffprobe. The fixture alternates typography shots with a 1600×1000 test photograph (enlarged, not reduced, at 1080p) under Ken Burns, pan and zoom moves, joined by 0.8 s dissolves, wipes and pushes. Output is 1080p30 at CRF 23. A 60-minute render drew all 296 segments in 71 minutes, then failed to upload because a signed URL issued at claim time had expired. URLs are now issued when the upload happens. Table 3 and Figure 6 show the subsequent two- and four-hour runs. Memory does not grow with segment count, and output duration matches the timeline exactly. Nearly every segment went to the GPU, consistent with the routing rule: every typography shot in

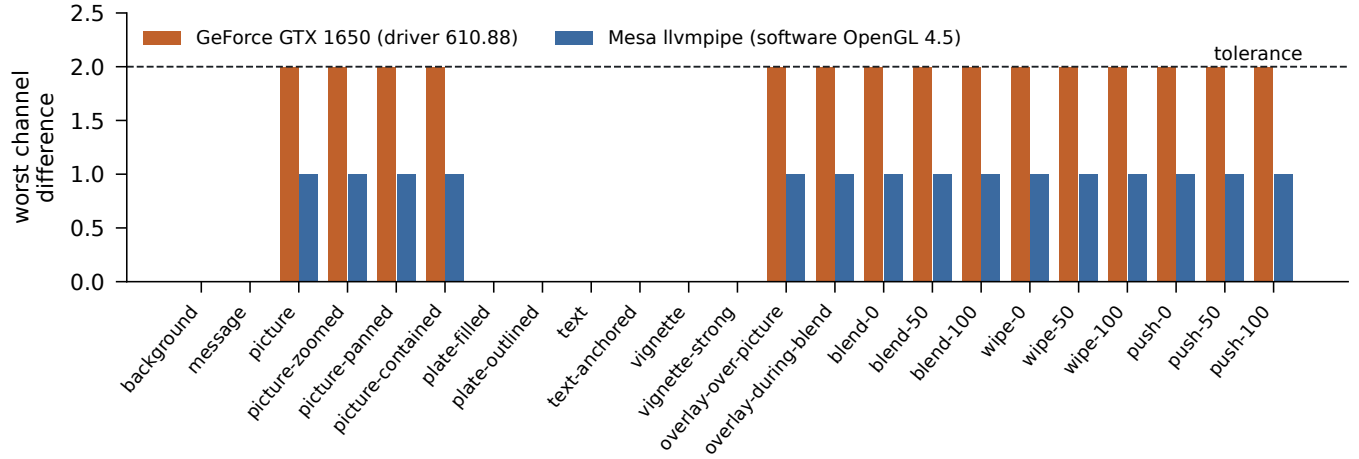


Figure 4: Worst per-channel difference from the CPU reference for all 23 calibration scenes (320×180, detailed still) on an NVIDIA GTX 1650 and on Mesa llvmpipe. Every scene has zero pixels outside tolerance on both.

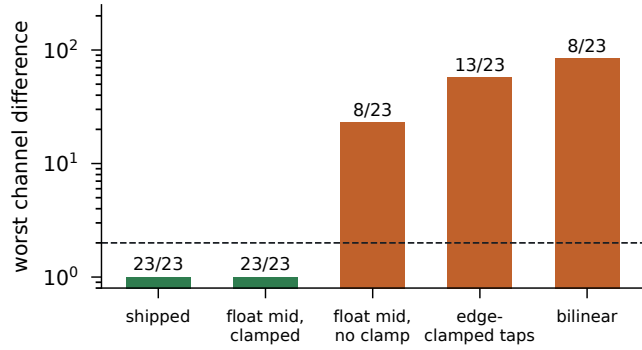


Figure 5: E4: worst channel difference (log scale) for each resampler variant on llvmpipe; labels give scenes passing the contract.

Table 3: Long renders on the GTX 1650 machine (1080p30).

Video	Render	Segs	GPU	RSS slope	Peak VRAM	Dur. err.
120 min	142.0 min	600	598	-0.095 MB/seg	505 MB	0.00 s
240 min	341.4 min	1,200	1,198	-0.113 MB/seg	916 MB	0.00 s

the fixture opens with a transition *from* a photograph. Peak scratch disk grew linearly with length.

8.5 GPU Throughput (E5)

I ran the benchmark kit (scripts/paper_bench.py) on the same Windows machine with the GTX 1650 (4 GB, driver 610.88), which has an Intel Comet Lake CPU (family 6, model 165) with 8 logical processors. For each of the four workloads from E1 it times composition on the CPU painter and on the GPU painter, and x264 encoding of the same frames, at 720p and 1080p. Each figure is the median of three repeats of 24 frames after one untimed warm-up

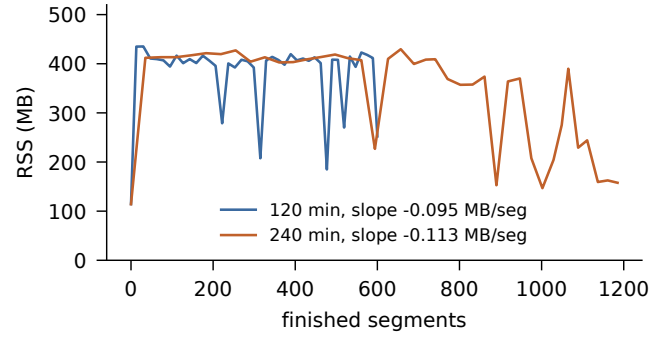


Figure 6: Resident memory against finished segments during the long renders (harness downsample of about 40 points).

Table 4: E5: composition frames/s on the CPU and GPU painters, and x264 encoding frames/s, on the GTX 1650 machine (medians of three repeats; GPU/CPU from unrounded values).

	Workload	CPU	GPU	GPU/CPU	x264
720p	typography	69.0	76.1	1.10×	115.7
	images	9.2	82.1	8.89×	56.1
	transitions	4.7	34.9	7.42×	44.5
	mixed	14.4	41.1	2.86×	45.8
1080p	typography	30.6	28.8	0.94×	48.4
	images	3.7	43.3	11.73×	24.2
	transitions	2.1	16.6	7.71×	26.8
	mixed	7.2	21.4	2.99×	22.6

frame. Table 4 and Figure 7 give the results; the raw times are in the repository.

Photographs are where the card pays: 11.7× at 1080p (10.0–11.9× across the three repeats), which agrees with the 11.6× I had written down in the code during development (the same note says

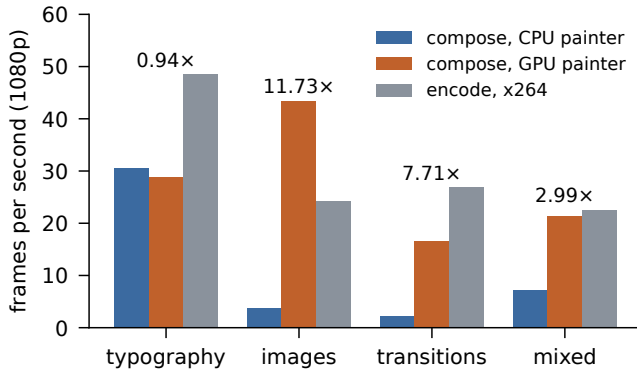


Figure 7: E5 at 1080p on the GTX 1650 machine: composition on the CPU and GPU painters, and x264 encoding. Labels give the GPU/CPU ratio.

typography was 2.2× slower; with the current painter I measured 0.94–1.10×). Transitions between photographs gain 7.4–7.7× and mixed content about 3×. Typography shows no reliable gain (1.10× at 720p, 0.94× at 1080p, noisy across repeats), because those frames are drawn by the animation engine on the CPU either way; the router already sends such segments to the processor. The gain for pictures grows with resolution, probably because resampling cost grows with pixel count and the card absorbs it better than one CPU core.

The profile after offloading. With the GPU painter, composition stops being the dominant cost for photographs. At 1080p, if each frame is composed and then encoded, composition falls from 87% of the work to 36% for images, from 93% to 62% for transitions and from 76% to 51% for mixed content, and x264 becomes the bigger half for images. The same Amdahl argument that ruled out a hardware encoder in §4 now points the other way: with photographs on the GPU, an encoder of infinite speed could make an images-only render up to 2.79× faster (1.95× for mixed content), against 1.15× and 1.32× before offloading on the same machine. Summing compose and encode time per frame overstates the total slightly, because in the renderer the two run in separate processes and overlap, but the direction does not change. Measuring NVENC against x264 per segment on this card is the natural next step.

9 Discussion

For a Python compositor feeding a software encoder, my results suggest an order of work. (i) Profile per content kind before choosing an accelerator. The popular fix, a hardware encoder, addresses the smaller share, and removing redundant buffer work gave 2.2× at no hardware cost. (ii) Parallelise across processes by segmenting on frame indices. One decomposition then yields resumability, parallelism and a unit of fallback, and exact partitioning preserves audio sync by construction. (iii) Move to the GPU only the operation that is both expensive and parallel, and keep the reference for everything that must match exactly. (iv) Treat correctness as a contract with a zero-outlier rule, and admit a device only after it passes a calibration frame at run time. The ablations show that the rule is sensitive where it needs to be. Dropping the inter-pass clamp,

clamping taps to the edge, or using the texture unit’s filter each fails it, by 23 to 84 levels. (v) Re-profile after each acceleration. On this machine the GPU made photographs 11.7× faster to compose, and that moved the bottleneck to the encoder.

10 Threats to Validity and Limitations

One machine for speed. E5 and the laptop half of E2 come from a single Windows machine with a GTX 1650, three repeats per figure, without controlling CPU frequency or thermal state. Typography timings in particular were noisy (GPU/CPU from 0.79× to 1.35× across repeats). Other GPUs will give different ratios. The long renders took 1.18× and 1.42× the video’s duration with the GPU painter, but they have no matched CPU-only baseline, so they are not a speed result. **Few devices.** Hardware GPU evidence comes from one GTX 1650. llvmpipe is an independent OpenGL implementation but not a GPU. AMD, Intel and macOS (where OpenGL is capped at 4.1) are untested. **Small calibration frames.** Equivalence was measured at 320×180 and 640×360. The long renders ran at 1080p but checked duration and memory, not pixels. **Synthetic workloads.** E1, E2, E5 and the long renders use synthetic content. The 77/23 project profile and the 12→26 frames/s result are recorded in the source and documentation, but the raw profiler output is not. **E1 hardware.** The per-content profile ran on a 2-vCPU cloud container; E5 repeats the CPU and x264 numbers on the GTX 1650 machine, and the composition shares agree within seven percentage points (61, 87, 93 and 76% at 1080p).

11 Conclusion

The fastest way to render this workload was not to reach for the GPU first. Encoding was the smaller cost, and much of composition was avoidable. Frame-exact segments made rendering resumable and parallel in one step. A narrow GPU resampler, admitted only when it reproduces the reference within two levels with no outliers, matches on two independent OpenGL implementations, and its inter-pass clamp, dropped edge taps and Lanczos kernel are each necessary. On a GTX 1650 it composes photographs 11.7× faster than the CPU at 1080p, and four hours of video rendered with flat memory and exact duration. The next bottleneck is the encoder.

Artifacts. The code is in the project repository [10]. The paper/folder holds the experiment scripts and raw results (E1–E5), the GTX 1650 diagnosis report and images, the long-render reports and this paper’s source. `scripts/paper_bench.py` produced E5 and the laptop half of E2.

Acknowledgments

I used AI coding and writing assistants while building the software and drafting this paper. I checked every number against the repository, and all measurements come from the project’s own scripts.

References

- [1] Gene M. Amdahl. 1967. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*. ACM, New York, NY, USA, 483–485. doi:10.1145/1465482.1465560
- [2] Lixiang Ao, Liz Izhikevich, Geoffrey M. Voelker, and George Porter. 2018. Sprocket: A Serverless Video Processing Framework. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. ACM, New York, NY, USA, 12 pages. doi:10.1145/3267809.3267815

- [3] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 5 (2015), 507–525.
- [4] Jeffrey A. Clark and contributors. 2026. Pillow (Python Imaging Library fork). <https://python-pillow.github.io>. Version 12.2.0 used in this work.
- [5] Jeffrey Dean and Sanjay Ghemawat. 2004. MapReduce: Simplified Data Processing on Large Clusters. In *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, San Francisco, CA, USA, 137–150.
- [6] Claude E. Duchon. 1979. Lanczos Filtering in One and Two Dimensions. *Journal of Applied Meteorology* 18, 8 (1979), 1016–1022.
- [7] E. N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. 2002. A Survey of Rollback-Recovery Protocols in Message-Passing Systems. *Comput. Surveys* 34, 3 (2002), 375–408.
- [8] FFmpeg developers. 2023. FFmpeg. <https://ffmpeg.org>. Version 6.1.1 used in this work.
- [9] Sadjad Fouladi, Riad S. Wahby, Brennan Shacklett, Karthikeyan Vasuki Balasubramaniam, William Zeng, Rahul Bhalerao, Anirudh Sivaraman, George Porter, and Keith Winstein. 2017. Encoding, Fast and Slow: Low-Latency Video Processing Using Thousands of Tiny Threads. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, Boston, MA, USA, 363–376.
- [10] Rajesh Kumar Kona. 2026. Voice-to-Video. <https://github.com/0KRK0/Voice-to-Video>.
- [11] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.
- [12] Don P. Mitchell and Arun N. Netravali. 1988. Reconstruction Filters in Computer Graphics. *ACM SIGGRAPH Computer Graphics* 22, 4 (1988), 221–228.
- [13] ModernGL contributors. 2026. ModernGL: Modern OpenGL binding for Python. <https://github.com/moderngl/moderngl>.
- [14] NVIDIA. 2026. Video Encode and Decode GPU Support Matrix. <https://developer.nvidia.com/video-encode-and-decode-gpu-support-matrix-new>. Accessed 25 September 2026.
- [15] Thomas Porter and Tom Duff. 1984. Compositing Digital Images. *ACM SIGGRAPH Computer Graphics* 18, 3 (1984), 253–259.
- [16] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, New York, NY, USA, 519–530. doi:10.1145/2491956.2462176
- [17] Mark Segal and Kurt Akeley. 2010. *The OpenGL Graphics System: A Specification (Version 3.3, Core Profile)*. The Khronos Group.
- [18] The Mesa 3D Graphics Library contributors. 2026. Mesa and the llvmpipe software rasteriser. <https://docs.mesa3d.org/drivers/llvmpipe.html>. Version 25.2.8 used in this work.
- [19] Tom’s Hardware. 2023. Nvidia Lifts Some Video Encoding Limitations from Consumer GPUs. <https://www.tomshardware.com/news/nvidia-increases-concurrent-nvenc-sessions-on-consumer-gpus>. 24 March 2023; accessed 25 September 2026.
- [20] Ken Turkowski. 1990. Filters for Common Resampling Tasks. In *Graphics Gems*, Andrew S. Glassner (Ed.). Academic Press, 147–165.
- [21] VideoLAN. 2026. x264: A free H.264/AVC encoder. <https://www.videolan.org/developers/x264.html>.
- [22] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. 2004. Image Quality Assessment: From Error Visibility to Structural Similarity. *IEEE Transactions on Image Processing* 13, 4 (2004), 600–612.
- [23] Nathan Whitehead and Alex Fit-Florea. 2011. *Precision & Performance: Floating Point and IEEE 754 Compliance for NVIDIA GPUs*. White paper. NVIDIA Corporation.